

TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE MILPA ALTA

Ingeniería en Sistemas Computacionales

Carrera

Programación Web

Nombre de la Asignatura

Ing. Luis Angel Fernández Blancas

Profesor

Solares Mendez Cristian Abdiel

García Flores Maribel

Alumno(a)

221070137

221070106

No de Control

Seguimiento No 4

Número de Seguimiento

Sistema de inventario

Nombre de la Actividad

18/05/2026

Fecha de entrega

Tabla de contenido

1. Introducción.....	4
2. Objetivos	6
Objetivo general.....	6
Objetivos específicos.....	6
3. Herramientas utilizadas.....	7
4. Arquitectura del sistema.....	8
4.1 Modelo	9
4.2 Vista.....	9
4.3 Controlador	12
5. Base de datos.....	17
6.1 Configuración XAMPP	20
6.2 Creación proyecto	20
6.3 Configuración conexión BD	21
6.4 Sistema login	26
7. Módulos implementados.....	27
7.1 Dashboard	27
7.2 Productos	27
7.3 Escáner código barras	28
7.4 Punto de venta	31
7.5 Usuarios.....	31
7.6 Permisos.....	32
7.7 Corte del día	32
7.8 Ganancias	33
7.9 Auditoría.....	33
8. Seguridad	34
8.1 Manejo de sesiones.....	34
8.2 Validación de acceso	35
8.3 Cierre de sesión.....	35
8.4 Bloqueo de regreso al login.....	36
8.5 Roles de usuario	37

8.6 Permisos por usuario.....	38
8.7 Protección de módulos	39
8.8 Beneficios de seguridad implementada.....	39
9. Acceso móvil	40
9.1 Procedimiento realizado	41
9.2 Beneficios obtenidos	42
9.3 Ventajas para desarrollo	42
9.4 Limitaciones	43
9.5 Posible mejora futura.....	43
10. Resultados obtenidos	44
11. Conclusiones.....	45
12. Mejoras futuras.....	47
Funcionalidades nuevas agregadas	48

1. Introducción

En la actualidad, muchos pequeños negocios y papelerías continúan llevando el control de inventario, ventas y ganancias de forma manual, utilizando libretas, hojas de cálculo o registros poco organizados. Esta forma de administración genera diversos problemas como pérdida de información, errores en el conteo de productos, dificultades para controlar existencias, desconocimiento de ganancias reales y retrasos en el proceso de venta.

Ante esta necesidad, se desarrolló un sistema web de inventario y punto de venta orientado específicamente a papelerías y pequeños negocios, con el propósito de automatizar y optimizar la gestión diaria de productos, ventas, usuarios y reportes administrativos.

El sistema permite registrar productos, actualizar existencias, realizar ventas, consultar ganancias, generar cortes del día y administrar usuarios con diferentes niveles de acceso mediante roles y permisos. De esta manera, el negocio puede mantener un control más preciso de su operación, reducir errores humanos y mejorar la toma de decisiones.

Adicionalmente, el proyecto incorpora funcionalidades modernas como acceso desde dispositivos móviles, autenticación segura mediante login, control de sesiones, permisos personalizados por usuario y lectura de códigos de barras mediante cámara, lo cual brinda una experiencia más eficiente y cercana a sistemas comerciales reales.

El desarrollo fue realizado utilizando tecnologías web como PHP, MySQL, HTML, CSS y JavaScript, implementando arquitectura MVC para mantener una estructura organizada, escalable y fácil de mantener.

Este proyecto busca ofrecer una solución tecnológica funcional, adaptable y económicamente accesible para negocios pequeños que requieren digitalizar sus procesos de inventario y ventas sin necesidad de invertir en sistemas costosos.

Sistema de Inventario

Ingresar

Sistema de Inventario - Papelería

Cerrar sesión

Bienvenido, Administrador

Rol: Administrador

Productos

Usuarios / Permisos

Ventas de empleados

Relación de ventas

Corte del día

Ganancias

Auditoría

Sistema de Inventario - Papelería

Bienvenido, cris

Rol: Empleado

Punto de venta

Productos

Corte del día

Ganancias

2. Objetivos

Objetivo general

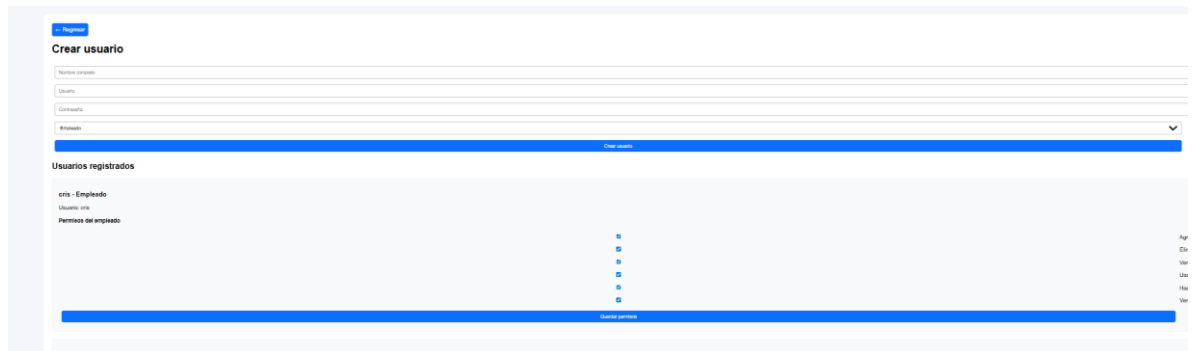
Desarrollar un sistema web para administrar inventario y ventas de una papelería.

Objetivos específicos

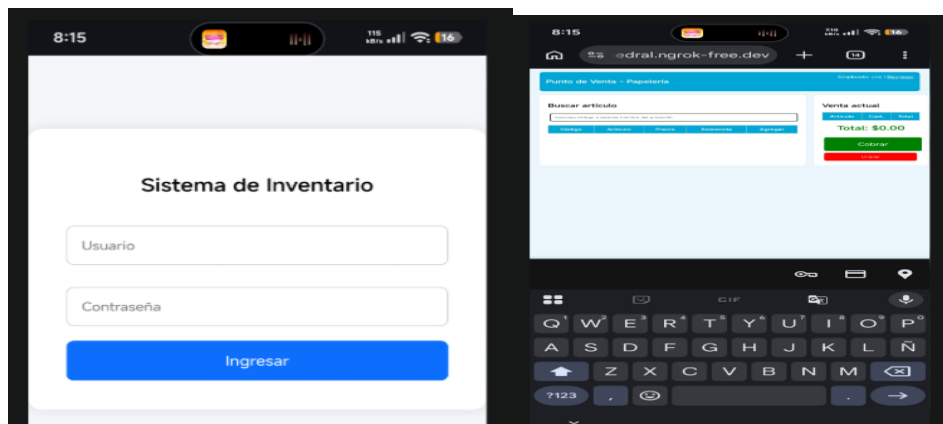
- controlar productos
- registrar ventas
- calcular ganancias
- administrar usuarios
- generar cortes diarios
- acceso

desde

celular



The screenshot shows a web application interface. At the top, there is a 'Registrar' button. Below it is a 'Crear usuario' (Create user) form with fields for 'Nombre completo', 'Usuario', 'Contraseña', and 'Email'. A 'Crear usuario' button is at the bottom of the form. Below the form is a section titled 'Usuarios registrados' (Registered users) which contains a table. The table has columns for 'Nombre completo', 'Usuario', 'Contraseña', 'Email', and 'Acciones'. The 'Acciones' column contains a 'Ver' button for each user. The table is currently empty.



The left screenshot shows a mobile application interface for a 'Sistema de Inventario' (Inventory System). It has a login screen with fields for 'Usuario' (Username) and 'Contraseña' (Password), and an 'Ingresar' (Login) button. The right screenshot shows a mobile application interface for a 'Punto de Venta - Papelería' (Point of Sale - Stationery). It has a search bar, a list of products, and a 'Venta actual' (Current Sale) section showing a total of \$0.00.

3. Herramientas utilizadas.

Herramienta	Uso
PHP	Backend
MySQL	Base de datos
HTML	Interfaz
CSS	Diseño
JavaScript	Interactividad
XAMPP	Servidor local
phpMyAdmin	Gestión BD
ngrok	acceso externo
VS Code	editor

4. Arquitectura del sistema.

Para el desarrollo del sistema de inventario y punto de venta se utilizó la arquitectura **MVC (Modelo - Vista - Controlador)**, un patrón de diseño ampliamente utilizado en aplicaciones web debido a su organización, mantenimiento y escalabilidad.

La arquitectura MVC divide el sistema en tres componentes principales:

- Modelo
- Vista
- Controlador

Esta separación permite organizar mejor el código y asignar responsabilidades específicas a cada parte del sistema.

Nombre	Fecha de Modificación	tipo
 assets	13/05/2026 11:21 p. m.	Carpeta de archivos
 controlador	16/05/2026 12:01 a. m.	Carpeta de archivos
 modelo	13/05/2026 11:13 p. m.	Carpeta de archivos
 vista	16/05/2026 11:17 a. m.	Carpeta de archivos

¿Qué es MVC?

MVC significa:

Modelo → Vista → Controlador

Cada componente cumple una función específica dentro del sistema.

4.1 Modelo

El **Modelo** se encarga de la lógica relacionada con datos y conexión a base de datos.

Responsabilidades:

- conectar con MySQL
- ejecutar consultas SQL
- insertar datos
- actualizar registros
- eliminar información

Archivo principal:

`conexion.php`

Ubicación:

`modelo/conexion.php`

Ejemplo de uso:

```
$conexion = new mysqli($host, $usuario, $password, $bd);
```

El modelo interactúa directamente con la base de datos:

`inventario_seguro`

4.2 Vista

La **Vista** corresponde a la interfaz gráfica mostrada al usuario.

Responsabilidades:

- mostrar formularios
- tablas
- botones
- dashboards
- reportes

Archivos implementados:

[login.php](#)

[dashboard.php](#)

[productos.php](#)

[ganancias.php](#)

[corte_dia.php](#)

[usuarios.php](#)

Ejemplos:

login.php

Permite autenticación del usuario.

Funciones:

- usuario
- contraseña
- iniciar sesión

dashboard.php

Panel principal del sistema.

Muestra:

- menú

- módulos
- accesos según rol

Bienvenido, Administrador

Rol: Administrador

Productos

Usuarios / Permisos

Ventas de empleados

Relación de ventas

Corte del día

Ganancias

Auditoría

productos.php

Gestión de inventario.

Funciones:

- agregar producto
- editar
- eliminar
- consultar stock

La vista es la capa visible del sistema.

[← Regresar](#)

Productos

Guardar producto

Productos registrados

Código / QR	Producto	Descripción	Proveedor	Compra	Venta	Stock	Acción
-------------	----------	-------------	-----------	--------	-------	-------	--------

4.3 Controlador

El **Controlador** actúa como intermediario entre Vista y Modelo.

Responsabilidades:

- recibir datos del usuario
- procesar formularios
- validar información
- ejecutar lógica de negocio

Archivos implementados:

`loginController.php`

`productoController.php`

`usuarioController.php`

`ventaController.php`

loginController.php

Funciones:

- validar credenciales
- iniciar sesión
- crear variables de sesión

productoController.php

Funciones:

- guardar productos

- editar productos
- eliminar productos

usuarioController.php

Funciones:

- crear usuarios
- asignar permisos
- guardar roles

Usuarios registrados

rosa - Empleado

Usuario: rosa

Permisos del empleado

Agregar productos



Eliminar productos



productos



Ver

punto de venta



Usar

corte del día



Hacer

ganancias



Ver

Guardar permisos

Flujo de funcionamiento

El flujo general del sistema sigue el patrón:

Vista → Controlador → Modelo → Base de datos

Proceso:

1. Vista

Usuario interactúa con formulario.

Ejemplo:

login.php

2. Controlador

Recibe datos.

Ejemplo:

loginController.php

Valida:

- usuario
- contraseña

3. Modelo

Realiza consulta.

Ejemplo:

conexion.php

Consulta MySQL.

4. Base de datos

Retorna información.

Ejemplo:

- usuarios

- productos
- ventas

Ventajas de usar MVC

La implementación de MVC permitió:

- mejor organización del código
- mantenimiento sencillo
- reutilización de archivos
- escalabilidad
- separación de responsabilidades

Beneficios prácticos:

- modificar diseño sin afectar lógica
- modificar consultas sin afectar interfaz

Estructura de carpetas

El proyecto quedó organizado de la siguiente manera:

proyecto/

|

├── assets/

├── controlador/

├── modelo/

└── vista/

Detalle:

assets

Recursos:

- CSS
- JS
- imágenes

controlador

Lógica del sistema.

modelo

Conexión BD.

vista

Interfaces visuales.

Resultado

El uso de arquitectura MVC permitió desarrollar un sistema estructurado, modular y mantenible, facilitando futuras mejoras y escalabilidad del proyecto.

Diagrama

Usuario



Vista



Controlador



Modelo



Base de datos

5. Base de datos.

Nombre BD: inventario_seguro

Tablas:

usuarios

Campos:

- id_usuario
- nombre
- usuario
- password
- id_rol
- estado

roles

- Administrador
- Empleado

productos

- id_producto
- codigo
- nombre_producto
- precio_compra
- precio_venta
- stock

ventas

- id_venta
- id_usuario
- fecha_venta

detalle_ventas

- id_detalle
- id_venta
- id_producto
- cantidad
- precio
- subtotal

usuario_permiso

- id_usuario
- permiso

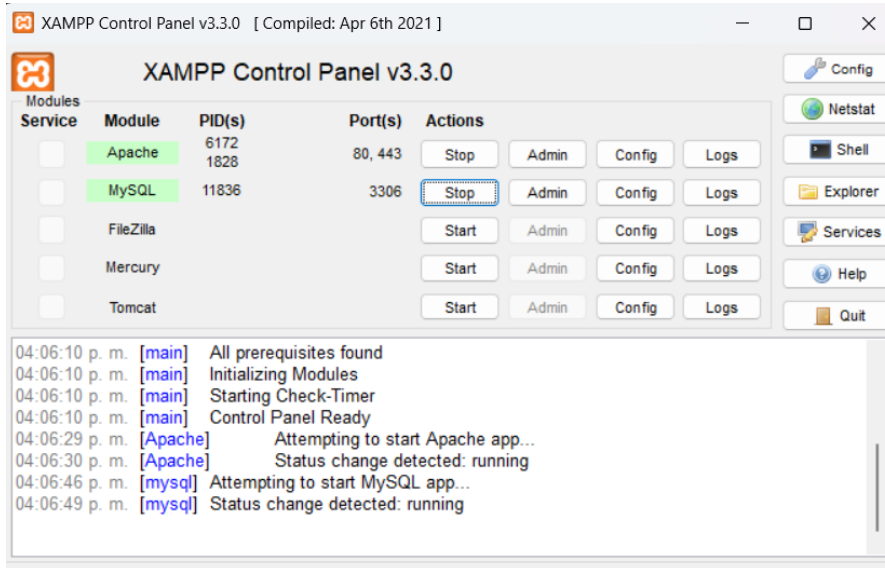
Opciones extra

19

6. Desarrollo paso a paso

6.1 Configuración XAMPP

- iniciar Apache
- iniciar MySQL



6.2 Creación proyecto

Ruta:

htdocs/proyecto

Estructura:

proyecto/

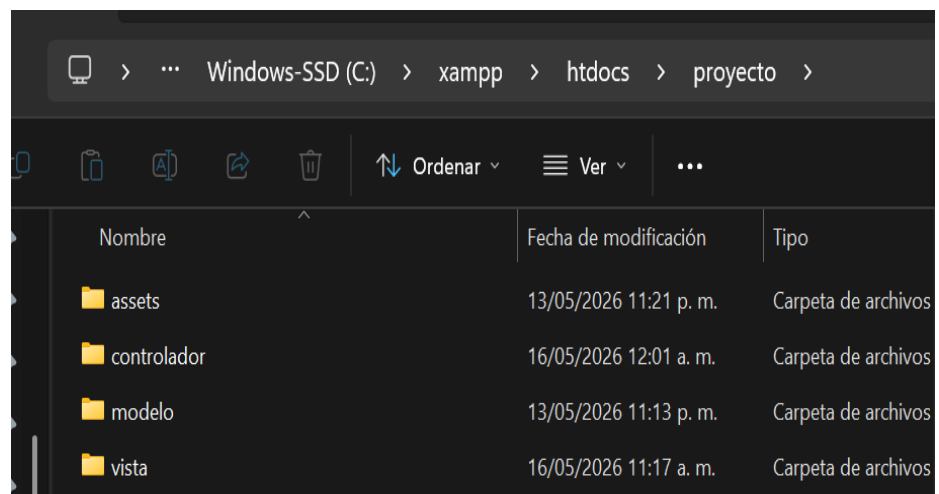
|

|— assets

|— controlador

|— modelo

|— vista



6.3 Configuración conexión BD

Conexión con MySQL usando MySQLi

La conexión a la base de datos permite que el sistema web interactúe con MySQL para almacenar, consultar, actualizar y eliminar información.

En este proyecto se utilizó **MySQLi** (MySQL Improved), una extensión de PHP diseñada para conectarse a bases de datos MySQL de manera segura y eficiente.

¿Qué es MySQLi?

MySQLi es una librería de PHP que permite:

- conectarse a MySQL
- ejecutar consultas SQL
- insertar registros
- actualizar datos
- eliminar información
- usar consultas preparadas para mayor seguridad

Archivo de conexión

Ruta:

modelo/conexion.php

Código utilizado:

```
<?php
```

```
$host = "localhost";
```

```
$usuario = "root";
```

```
$password = "";  
  
$bd = "inventario_seguro";  
  
$conexion = new mysqli($host, $usuario, $password, $bd);  
  
if ($conexion->connect_error) {  
    die("Error de conexión: " . $conexion->connect_error);  
}  
  
$conexion->set_charset("utf8");  
  
?>
```

Explicación del código

Variables de conexión

```
$host = "localhost";
```

Define el servidor donde está alojada la base de datos.

En XAMPP normalmente se usa:

```
localhost
```

porque la base de datos corre localmente.

```
$usuario = "root";
```

Usuario de MySQL.

En XAMPP por defecto:

```
root
```

```
$password = "";
```

Contraseña del usuario MySQL.

En local suele estar vacía.

```
$bd = "inventario_seguro";
```

Nombre de la base de datos utilizada por el sistema.

Crear conexión

```
$conexion = new mysqli($host, $usuario, $password, $bd);
```

Crea un objeto MySQLi con:

- servidor
- usuario
- contraseña
- base de datos

Si todo es correcto, la conexión queda activa.

Validar errores

```
if ($conexion->connect_error) {  
    die("Error de conexión: " . $conexion->connect_error);  
}
```

Verifica si ocurrió un error.

Si falla:

- servidor apagado
- usuario incorrecto
- contraseña incorrecta
- BD inexistente

el sistema se detiene y muestra el error.

Configurar codificación UTF-8

```
$conexion->set_charset("utf8");
```

Permite almacenar caracteres especiales correctamente:

- ñ
- acentos
- símbolos

Ejemplo:

papelería

información

código

Importancia en el sistema

Esta conexión se usa en todos los módulos:

- login
- productos
- ventas
- usuarios
- permisos
- ganancias
- corte del día

Ejemplo:

```
include("../modelo/conexion.php");
```

Esto reutiliza la misma conexión en todo el sistema.

Ventajas de usar MySQLi

- mayor seguridad
- consultas preparadas
- rapidez
- compatibilidad con PHP
- control de errores
-

Resultado

Gracias a MySQLi, el sistema puede:

- registrar ventas

- guardar productos
- validar usuarios
- calcular ganancias
- consultar reportes

de manera dinámica y segura.

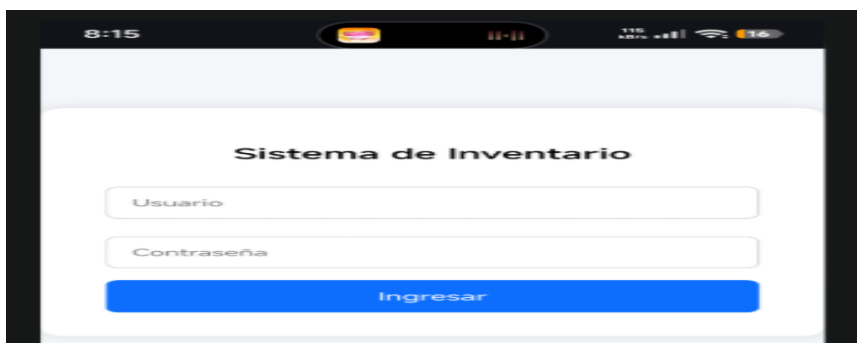
6.4 Sistema login

Archivos:

- login.php
- loginController.php
- seguridad.php
- logout.php

Funciones:

- iniciar sesión
- validar usuario
- cerrar sesión



The screenshot shows a mobile application interface for a 'Sistema de Inventario'. At the top, the status bar displays the time 8:15, a battery icon, and signal strength. The app's title bar is black with a red and yellow logo. The main content area is white and contains the title 'Sistema de Inventario' in bold. Below the title are two input fields: 'Usuario' and 'Contraseña'. At the bottom of the form is a blue button labeled 'Ingresar'.

7. Módulos implementados

7.1 Dashboard

Funciones:

- menú principal
- acceso por rol

Bienvenido, Administrador

Rol: Administrador

[Productos](#) [Usuarios / Permisos](#) [Ventas de empleados](#) [Relación de ventas](#) [Corte del día](#) [Ganancias](#) [Auditoría](#)

7.2 Productos

Funciones:

- agregar producto
- editar
- eliminar
- stock
- código

[← Regresar](#)

Productos

[Guardar producto](#)

Productos registrados

Código / QR	Producto	Descripción	Proveedor	Compra	Venta	Stock	Acción
-------------	----------	-------------	-----------	--------	-------	-------	--------

7.3 Escáner código barras

Con el objetivo de agilizar el registro de productos y ventas dentro del sistema, se integró un módulo de lectura de códigos de barras mediante cámara web o cámara de dispositivo móvil.

Esta funcionalidad permite capturar automáticamente el código de barras de un producto sin necesidad de escribirlo manualmente, reduciendo errores humanos y acelerando el proceso de venta.

Funcionamiento:

El sistema utiliza la cámara del dispositivo para detectar y leer códigos de barras en tiempo real.

Proceso:

1. El usuario ingresa al módulo de ventas o productos.
2. Presiona el botón **Escanear código**.
3. Se activa la cámara del dispositivo.
4. El sistema detecta el código de barras del producto.
5. El código leído se inserta automáticamente en el campo correspondiente.
6. Se consulta la base de datos para localizar el producto.

Si el producto existe:

- muestra nombre
- precio
- stock
- descripción

Si no existe:

- permite registrarlo como nuevo producto.

Uso en productos:

En el módulo de productos, el escáner permite registrar rápidamente:

- código del producto
- búsqueda automática
- validación de existencia

Ejemplo:

7501031311309

Este código se guarda en el campo:

codigo

de la tabla productos.

Uso en ventas:

En el punto de venta, el escáner facilita:

- agregar productos al carrito
- consultar precios
- registrar venta inmediata

Flujo:

Escanear → Buscar producto → Agregar al carrito → Cobrar

Tecnologías utilizadas.

Para implementar esta funcionalidad se utilizaron:

- JavaScript
- HTML5
- cámara del navegador
- librería de lectura de códigos

Ejemplo de librería:

QuaggaJS / Html5-QRCode

Beneficios:

La integración del escáner proporciona:

- reducción de tiempo en ventas
- menor captura manual
- menos errores de digitación
- mayor control de inventario
- experiencia similar a sistemas comerciales

Ventajas para papelería:

Debido a que una papelería maneja gran cantidad de productos pequeños como:

- lápices
- plumas
- cuadernos
- colores
- pegamentos
- carpetas

el escáner permite una venta más rápida y ordenada.

Resultado obtenido:

Se logró integrar exitosamente el escaneo mediante cámara, permitiendo registrar y vender productos usando lectura automática de código de barras desde computadora o celular.

Puedes acompañarlo con captura de:

- botón escanear
- cámara abierta
- código detectado
- producto agregado al carrito

7.4 Punto de venta

Funciones:

- vender producto
- subtotal
- total
- cambio

Punto de Venta - Papelería

Empleado: cris | Regresar

Buscar artículo

Código	Artículo	Precio	Existencia	Agregar
--------	----------	--------	------------	---------

Venta actual

Artículo	Cant.	Total
Total: \$0.00		
Cobrar		
Limpiar		

7.5 Usuarios

Administrador puede:

- crear usuarios
- asignar roles
- permisos

7.6 Permisos

Permisos:

- agregar_producto
- eliminar_producto
- ver_productos
- punto_venta
- corte_dia
- ver_ganancias

7.7 Corte del día

Funciones:

- ventas diarias
- total vendido
- piezas vendidas
- ganancias

Corte del día

Selecciona fecha:

05/16/2026

Consultar

Imprimir corte

Fecha: 2026-05-16

Ventas 0	Piezas vendidas 0	Total vendido \$0.00	Ganancia total \$0.00
-------------	----------------------	-------------------------	--------------------------

Resumen por empleado

Empleado	Usuario	Ventas	Piezas	Total vendido	Ganancia
----------	---------	--------	--------	---------------	----------

Detalle de ventas

Folio	Empleado	Producto	Cantidad	Precio	Total	Ganancia	Hora
-------	----------	----------	----------	--------	-------	----------	------

7.8 Ganancias

Muestra:

- ganancia total
- ganancia por producto
- ganancia por empleado

7.9 Auditoría

Registro de:

- ventas
- productos
- usuarios

Auditoría del sistema

Usuario	Acción	Tabla	Descripción	Fecha y hora	IP	Sistema operativo
admin	LOGIN	usuarios	Inicio de sesión exitoso	2026-05-16 20:33:51	::1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36
cris	LOGIN	usuarios	Inicio de sesión exitoso	2026-05-16 20:32:08	::1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36
cris	LOGIN	usuarios	Inicio de sesión exitoso	2026-05-16 20:30:02	::1	Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Mobile Safari/537.36
rosa	LOGIN	usuarios	Inicio de sesión exitoso	2026-05-16 20:23:55	::1	Mozilla/5.0 (Linux; Android 15; CPH2531 Build/AP3A.240617.008; wv) AppleWebKit/537.36 (KHTML, like Gecko) Version/4.0 Chrome/148.0.7778.120 Mobile Safari/537.36 [FB_IAB/FB4A;FBAV/561.0.0.55.76;]
cris	LOGIN	usuarios	Inicio de sesión exitoso	2026-05-16 20:15:21	::1	Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Mobile Safari/537.36
admin	LOGIN	usuarios	Inicio de sesión exitoso	2026-05-16 20:13:41	::1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36
cris	LOGIN	usuarios	Inicio de sesión exitoso	2026-05-16 20:12:22	::1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36
admin	LOGIN	usuarios	Inicio de sesión exitoso	2026-05-16 20:11:40	::1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36
cris	LOGIN	usuarios	Inicio de sesión exitoso	2026-05-16 15:49:48	::1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36

8. Seguridad

La seguridad del sistema fue implementada para proteger el acceso, restringir funciones y evitar manipulación no autorizada del sistema de inventario y punto de venta.

Se implementaron mecanismos de autenticación, control de sesiones y permisos por rol.

8.1 Manejo de sesiones

Se utilizó PHP Sessions para mantener activa la sesión del usuario autenticado.

Código utilizado:

```
session_start();
```

Función

Permite almacenar información temporal del usuario después del login.

Datos almacenados:

- id_usuario
- nombre
- usuario
- rol

Ejemplo:

```
$_SESSION['id_usuario']
```

```
$_SESSION['nombre']
```

```
$_SESSION['rol']
```

Esto permite identificar al usuario dentro de todo el sistema.

8.2 Validación de acceso

Cada módulo verifica si existe sesión activa antes de permitir acceso.

Archivo:

controlador/seguridad.php

Código:

```
<?php
session_start();

if (!isset($_SESSION['id_usuario'])) {
    header("Location: ../vista/login.php");
    exit();
}
?>
```

Función

Si no existe sesión:

- bloquea acceso
- redirige al login

Evita entrar manualmente por URL.

Ejemplo:

localhost/proyecto/vista/dashboard.php

sin iniciar sesión.

8.3 Cierre de sesión

Se implementó cierre seguro de sesión.

Archivo:

logout.php

Código:

```
session_start();
```

```
session_destroy();  
header("Location: ../vista/login.php");  
exit();
```

Función

Al cerrar sesión:

- elimina datos de sesión
- regresa al login

8.4 Bloqueo de regreso al login

Se implementó bloqueo para evitar regresar con la flecha del navegador después de iniciar sesión.

Problema:

Después del login, algunos navegadores permitían regresar con:

← atrás

mostrando nuevamente el login.

Solución 1: Deshabilitar caché

Código:

```
header("Cache-Control: no-cache, no-store, must-revalidate");  
header("Pragma: no-cache");  
header("Expires: 0");
```

Función

Evita que el navegador almacene páginas protegidas.

Solución 2: Bloqueo con JavaScript

Código:

```
<script>  
history.pushState(null, null, location.href);
```

```
window.onpopstate = function () {  
    history.go(1);  
};  
</script>
```

Función

Impide regresar con flecha atrás al login.

Beneficio:

- mayor seguridad
- mejor experiencia de usuario

8.5 Roles de usuario

El sistema maneja control de acceso mediante roles.

Tabla:

roles

Roles disponibles:

Administrador

Acceso total:

- productos
- usuarios
- ventas
- ganancias
- auditoría
- corte del día
- permisos

Empleado

Acceso limitado según permisos.

Ejemplo:

- vender

- ver productos
- corte del día
- ganancias

8.6 Permisos por usuario

Además del rol, se implementó control granular de permisos.

Tabla:

usuario_permiso

Permisos:

- agregar_producto
- eliminar_producto
- ver_productos
- punto_venta
- corte_dia
- ver_ganancias

Validación de permisos

Archivo:

permisos.php

Código:

```
tienePermiso($conexion, $id_usuario, "punto_venta")
```

Ejemplo:

```
if (tienePermiso($conexion, $_SESSION['id_usuario'], "ver_ganancias")) {  
    echo "Acceso permitido";  
}
```

8.7 Protección de módulos

Cada vista valida:

- sesión activa
- rol
- permiso

Ejemplo:

```
include("../controlador/seguridad.php");
```

```
include("../controlador/permisos.php");
```

Esto protege módulos como:

- productos
- ganancias
- usuarios
- corte del día

8.8 Beneficios de seguridad implementada

La seguridad implementada permite:

- acceso controlado
- bloqueo de usuarios no autenticados
- protección de módulos
- sesiones seguras
- permisos personalizados

Resultado

Se logró implementar un sistema seguro con autenticación, control de sesiones, bloqueo de caché y permisos dinámicos por usuario.

Esto garantiza que solo personal autorizado pueda acceder a funciones específicas del sistema.

9. Acceso móvil

Con el fin de permitir el acceso al sistema desde dispositivos móviles y realizar pruebas externas, se utilizó la herramienta **ngrok**.

Ngrok permite exponer un servidor local a internet mediante una URL pública temporal.

Esto hizo posible acceder al sistema de inventario desde cualquier dispositivo, incluyendo celulares y tablets.

¿Qué es ngrok?

Ngrok es una herramienta que crea un túnel seguro entre internet y el servidor local.

En este proyecto se utilizó para exponer el servidor local de XAMPP.

Servidor local:

localhost

Proyecto:

<http://localhost/proyecto>

Ngrok transforma esta dirección local en una URL pública.

Ejemplo:

<https://abc123.ngrok-free.app>

Comando utilizado

Para exponer el proyecto se ejecutó:

`ngrok http 80`

Explicación

- http indica protocolo web
- 80 es el puerto utilizado por Apache en XAMPP

Apache corre normalmente en:

<http://localhost:80>

9.1 Procedimiento realizado

1. Iniciar XAMPP

Se activaron:

- Apache
- MySQL

2. Abrir terminal

Se ejecutó ngrok desde consola.

Comando:

`ngrok http 80`

3. Obtener URL pública

Ngrok genera una URL similar a:

<https://1234abcd.ngrok-free.app>

Esta URL permite abrir el proyecto desde cualquier navegador.

4. Acceso al sistema

Se ingresó desde celular:

<https://1234abcd.ngrok-free.app/proyecto>

o

<https://1234abcd.ngrok-free.app/proyecto/vista/login.php>

ESTE FUE NUESTRO URL: <https://uncloak-system-cathedral.ngrok-free.dev/proyecto/vista/login.php>

9.2 Beneficios obtenidos

El uso de ngrok permitió:

Abrir desde celular

Acceso al sistema desde:

- Android
- iPhone
- tablet

Sin necesidad de estar dentro del proyecto local.

Pruebas remotas

Se pudieron realizar pruebas como:

- login móvil
- ventas desde celular
- dashboard responsivo
- escáner de cámara

Compartir proyecto

Se puede compartir el sistema con:

- clientes
- profesores
- testers

mediante URL temporal.

9.3 Ventajas para desarrollo

Ngrok facilitó:

- pruebas sin hosting
- acceso externo
- demostraciones en tiempo real

3.4 Limitaciones

Versión gratuita:

- URL cambia al reiniciar
- tiempo limitado

Ejemplo:

ngrok genera una nueva URL cada vez que se ejecuta.

9.5 Posible mejora futura

Subir el proyecto a hosting real para acceso permanente.

Opciones:

- InfinityFree
- Hostinger
- Railway

Resultado

Se logró acceder exitosamente al sistema de inventario desde dispositivos móviles mediante una URL pública generada por ngrok.

Esto permitió validar funcionamiento remoto y compatibilidad móvil del sistema.

10. Resultados obtenidos

Lista:

- sistema funcional
- control inventario
- control ventas
- ganancias
- acceso móvil
- usuarios y permisos

11. Conclusiones

Durante el desarrollo de este proyecto se logró diseñar, implementar y probar un sistema web de inventario y punto de venta enfocado principalmente en papelerías y pequeños negocios que requieren un mejor control de sus operaciones diarias.

El sistema desarrollado permite automatizar procesos que comúnmente se realizan de manera manual, tales como el registro de productos, control de existencias, actualización de stock, gestión de usuarios, control de ventas y generación de reportes financieros. Esto representa una mejora significativa en la organización interna del negocio, disminuyendo errores humanos, pérdida de información y tiempos de operación.

A lo largo del desarrollo se implementaron diferentes módulos funcionales, entre los cuales destacan el sistema de autenticación de usuarios, manejo de sesiones, control de roles y permisos, gestión de productos, punto de venta, cálculo de ganancias, corte del día, auditoría y acceso móvil. Cada uno de estos módulos fue diseñado con el objetivo de brindar seguridad, facilidad de uso y eficiencia operativa.

La implementación de roles diferenciados entre administrador y empleado permitió establecer restricciones adecuadas para proteger información sensible del negocio, otorgando únicamente acceso a funcionalidades autorizadas según el perfil del usuario. Asimismo, la asignación dinámica de permisos mejora la flexibilidad del sistema y lo hace adaptable a distintos modelos de negocio.

Otro aspecto relevante fue la integración del escáner de código de barras mediante cámara, funcionalidad que incrementa la rapidez en el registro de productos y ventas, acercando el sistema a soluciones comerciales utilizadas en negocios reales.

En términos financieros, el sistema permite visualizar ganancias generales, ganancias por producto y ganancias por empleado, proporcionando información clave para la toma de decisiones administrativas. De igual forma, el módulo de corte del día facilita el monitoreo diario de ventas, ingresos y rendimiento operativo.

Se logró además habilitar acceso remoto mediante herramientas como ngrok, permitiendo realizar pruebas desde dispositivos móviles y validar la adaptabilidad del sistema en diferentes resoluciones de pantalla. Esto representa una ventaja importante para futuras implementaciones orientadas a movilidad o administración remota.

Desde el punto de vista técnico, el proyecto consolidó conocimientos relacionados con desarrollo web utilizando PHP, MySQL, HTML, CSS y JavaScript, así como conceptos de arquitectura MVC, gestión de bases de datos, seguridad web y despliegue de aplicaciones.

Finalmente, este sistema constituye una solución funcional, escalable y comercializable, capaz de ser utilizado como base para futuros desarrollos o mejoras, tales como implementación de impresión de tickets, sistema multi-sucursal, generación de facturas, aplicación móvil nativa y despliegue en servidores cloud o hosting profesional.

12. Mejoras futuras

Agregar:

- app Android
- impresión tickets
- QR pagos
- nube
- multi sucursal
- notificaciones stock

Funcionalidades nuevas agregadas

1. Aplicación Android con Android Studio

Se desarrolló una aplicación móvil utilizando Android Studio y WebView para acceder al sistema desde teléfonos Android.

Funciones:

- acceso desde celular
- compatibilidad móvil
- navegación responsive
- botón regresar
- impresión desde Android
- permisos de cámara

2. Escáner de código de barras

Se integró lectura de códigos mediante cámara.

Mejoras:

- escaneo automático
- búsqueda rápida de productos
- inserción automática del código
- compatible con Android y navegador

Tecnologías:

- JavaScript
- cámara del dispositivo
- Html5-QRCode / QuaggaJS

3. Impresión de tickets

Se agregó impresión automática de tickets después de finalizar una venta.

Funciones:

- ticket PDF/impresión
- impresión desde navegador
- impresión desde aplicación Android
- botón imprimir ticket
- vista ticket personalizada

4. Corte del día

Se agregó módulo administrativo de corte diario.

Muestra:

- total vendido
- ganancias
- productos vendidos
- ventas realizadas
- resumen financiero

5. Sistema de permisos

Se mejoró la seguridad mediante permisos dinámicos.

Permisos:

- agregar productos
- eliminar productos
- ver ganancias
- acceso a ventas
- acceso a corte del día

6. Seguridad del sistema

Se implementaron mejoras de seguridad.

Incluye:

- manejo de sesiones
- bloqueo de acceso directo
- cierre seguro de sesión
- bloqueo del botón atrás
- validación de usuarios

7. Acceso remoto mediante ngrok

El sistema puede abrirse desde internet usando ngrok.

Beneficios:

- acceso desde celular
- pruebas remotas
- demostraciones
- conexión externa sin hosting

8. Compatibilidad responsive

Se adaptó el sistema para:

- horizontal
- vertical
- celulares
- tablets
- computadoras

9. Integración de impresión Android

Se agregó soporte para impresión desde la aplicación móvil usando:

PrintManager

createPrintDocumentAdapter()

10. Mejoras en base de datos

Se agregaron:

- relaciones foráneas
- control de stock
- estados de productos

- auditoría
- validaciones SQL

Esto también puedes agregar en “Resultados obtenidos”

Resultados nuevos

- sistema funcional en Android
- impresión de tickets funcionando
- acceso remoto desde celular
- escáner operativo
- control de permisos
- corte del día funcional
- seguridad mejorada
- compatibilidad móvil completa

También puedes agregar en “Mejoras futuras”

Mejoras futuras nuevas

- hosting profesional
- APK instalable
- notificaciones push
- sincronización cloud
- facturación electrónica
- multiusuario en tiempo real
- impresora térmica Bluetooth
- aplicación offline